



# Security Review For Layeredge



Public Audit Contest Prepared For: **Layeredge**  
Lead Security Expert: **X77**  
Date Audited: **May 22 - May 25, 2025**  
Final Commit: **edefalb**

## Introduction

LayerEdge is a Bitcoin backed Internet using trust minimized verification & proof aggregation for all layers & apps. This contest focuses on staking contract that reward users as per FCFS model where earlier stakers receive higher rewards.

## Scope

Repository: Layer-Edge/edgen-staking

Audited Commit: 3a7f98da9d5dba030823249ff1d2c623d16cbae2

Final Commit: edefa1b365bbc39a2dd8aeabe4c4bde0069eb8ca

Files:

- src/WETH9.sol
- src/library/FenwickTree.sol
- src/stake/LayerEdgeStaking.sol

## Final Commit Hash

edefa1b365bbc39a2dd8aeabe4c4bde0069eb8ca

## Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.

## Issues Found

| High | Medium |
|------|--------|
| 2    | 1      |

## Issues Not Fixed and Not Acknowledged

| High | Medium |
|------|--------|
| 0    | 0      |

## Security experts who found valid issues

[0x15](#)

[AestheticBhai](#)

[Artur](#)

[Bigsam](#)

[Boy2000](#)

[Cybrid](#)

[Drynooo](#)

[Galturok](#)

[Hecok](#)

[Hurricane](#)

[KobbyEugene](#)

[MysteryAuditor](#)

[Ollam](#)

[Pablo](#)

[PersonaNormale](#)

[X77](#)

[ZafiN](#)

[algiz](#)

[benjamin\\_0923](#)

[durov](#)

[eLSeR17](#)

[future](#)

[gesha17](#)

[illoy\\_sci](#)

[jkoppel](#)

[kazan](#)

[lukris02](#)

[montecristo](#)

[newspacexyz](#)

[novaman33](#)

[ptsanev](#)

[rudhra1749](#)

[shiazinho](#)

[sil3th](#)

[slavina](#)

[t0x1c](#)

[tjonair](#)

[tobi0x18](#)

[wickie](#)

# Issue H-1: Incorrect tier tracking when tier 3 staker exits in a certain case

Source: <https://github.com/sherlock-audit/2025-05-layeredge-judging/issues/83>

## Found by

MysteryAuditor, X77, future, jkoppel, newspacexyz, rudhral749, slavina, wickie

## Summary

Incorrect tier tracking when tier 3 staker exits in a certain case

## Root Cause

```
else if (isRemoval) {  
    _findAndRecordTierChange(new_t1 + new_t2, n);  
}
```

Whenever a staker exits and the old T2 and new T2 counts are the same, then we'd end up above. However, this code does not properly work when a T3 staker exits from the tree and completely breaks the tier tracking.

In our scenario below (attack path and POC), there are 7 T1 and T2 and 8 T3 stakers (15 total) stakers initially. T3 staker exits, so now there must be 2 T1 stakers, 4 T2 stakers and 8 T3 stakers where the last T2 staker is demoted. However, in the snippet shown above, we write to `new_t1 + new_t2` rank which is 6 (2 + 4). However, as a T3 staker exited, then the 7th rank is the actual last T2 staker, not the 6th we are writing to.

## Internal Pre-conditions

- Check in the root cause snippet should be reached, this happens when the T2 count before and after a removal are the same

## External Pre-conditions

•

## Attack Path

1. There are 15 stakers, distribution is:

- $T1 = 15 * 0.2 = 3$

- $T2 = 15 * 0.3 = 4$
- $T3 = 15 - 3 - 4 = 8$

2. User from T3 unstakes, distribution should be:

- $T1 = 14 * 0.2 = 2$
- $T2 = 14 * 0.3 = 4$
- $T3 = 14 - 4 - 2 = 8$

3. The following code is reached upon the removal and the T2 boundary handling block:

```
else if (isRemoval) {
    _findAndRecordTierChange(new_t1 + new_t2, n);
}
```

4. We write to the 6th rank (2 + 4), however as a T3 staker exited, then the current T1 and T2 staker counts are still the initial 7 (3 + 4), thus we must write to rank 7 (if T1 or T2 exits, then the 6th rank is properly demoted as that is the correct boundary as T1 and T2 current counts are now 6 due to the exit, however here, 7th is the correct boundary as T3 exited and T1 and T2 counts didn't change)
5. As we write to the 6th rank, he remains T2 instead of the 7th rank demoting to T3, thus the distribution breaks and is now 2 T1 (correct), 5 T2 (incorrect), 7 T3 (incorrect)

## Impact

Incorrect tier tracking, loss of funds for the protocol as they must handle a bigger APY due to more users being in T2 unfairly

## PoC

First, due to issue starting with title An edge case in . . . ., we must do a little fix to reach the proper state:

```
    } else if (!isRemoval) {
+        _findAndRecordTierChange(new_t1 + new_t2, n);
-        _findAndRecordTierChange(old_t1 + old_t2, n);
    }
```

Note that I am not sure whether this fix is perfect, however it makes the state consistent, at least in the flow for our issue (we have assertions for that in our POC below).

Then, add this function in the staking contract to easily get the tier of a user:

```

function getTier(address user) public view returns (uint256) { // note: added
    if (stakerTierHistory[user].length == 0) return 0;
    TierEvent memory te = stakerTierHistory[user][stakerTierHistory[user].length -
↪ 1];
    return uint256(te.to);
}

```

Now, paste the following POC in the staking test contract:

```

function testIssue() public {
    for (uint256 i; i < 15; i++) {
        address user = address(uint160(i + 1));
        deal(address(token), user, 3000e18);
        vm.startPrank(user);
        token.approve(address(staking), MIN_STAKE);
        staking.stake(MIN_STAKE);
    }

    uint256 t1Count;
    uint256 t2Count;
    uint256 t3Count;
    for (uint256 i; i < 15; i++) {
        address user = address(uint160(i + 1));
        uint256 tier = staking.getTier(user);
        if (tier == 1) {
            t1Count++;
            continue;
        }

        if (tier == 2) {
            t2Count++;
            continue;
        }

        if (tier == 3) {
            t3Count++;
            continue;
        }
    }

    // NOTE: Changed `old_t1 + old_t2` to `new_t1 + new_t2` in the last block of
↪ `_checkBoundariesAndRecord()` so we get into a correct state due to another
↪ issue (not sure if this is the proper fix but it works in this case)
    assertEq(staking.stakerCountInTree(), 15);
    (uint256 t1ShouldBe, uint256 t2ShouldBe, uint256 t3ShouldBe) =
↪ staking.getTierCountForStakerCount(staking.stakerCountInTree());

    assertEq(t1Count, 3);
}

```

```

assertEq(t1ShouldBe, t1Count);
assertEq(t2Count, 4);
assertEq(t2ShouldBe, t2Count);
assertEq(t3Count, 8);
assertEq(t3ShouldBe, t3Count);

address unstakingUser = address(uint160(10));
uint256 r = staking.getTier(unstakingUser);
assertEq(r, 3);

vm.startPrank(unstakingUser);
staking.unstake(MIN_STAKE);

assertEq(staking.stakerCountInTree(), 14);
(t1ShouldBe, t2ShouldBe, t3ShouldBe) =
↪ staking.getTierCountForStakerCount(staking.stakerCountInTree());

t1Count = 0;
t2Count = 0;
t3Count = 0;
for (uint256 i; i < 15; i++) {
    address user = address(uint160(i + 1));
    if (unstakingUser == user) continue; // He left the tree above so shouldn't
↪ include him
    uint256 tier = staking.getTier(user);
    if (tier == 1) {
        t1Count++;
        continue;
    }

    if (tier == 2) {
        t2Count++;
        continue;
    }

    if (tier == 3) {
        t3Count++;
        continue;
    }
}

assertEq(t1Count, 2); // Correct!
assertEq(t1Count, t1ShouldBe);

assertEq(t2Count, 5); // Incorrect!
assertNotEq(t2Count, t2ShouldBe);

assertEq(t3Count, 7); // Incorrect
assertNotEq(t3Count, t3ShouldBe);

```

```
}
```

## Mitigation

*No response*

## Discussion

**sherlock-admin2**

The protocol team fixed this issue in the following PRs/commits:  
<https://github.com/Layer-Edge/edgen-staking/pull/7>



# Issue H-2: When stakerCountInTree Increases, Some Users May Receive Less Interest

Source: <https://github.com/sherlock-audit/2025-05-layeredge-judging/issues/230>

## Found by

Cybrid, Hecok, Hurricane, Pablo, X77, benjamin\_0923, durov, future, jkoppel, kazan, lukris02, montecristo, rudhral749, shiazinho, slavina, t0xlc

## Summary

In the `_checkBoundariesAndRecord` function, when `stakerCountInTree` increases, a user in tier 3 is not promoted to tier 2 when it should be.

## Root Cause

The root cause is that when `stakerCountInTree` increases, users who need to transition from tier 3 to tier 2 are not properly updated. At this point, the ranking of `new_t1 + new_t2` could be moved from tier 3 to tier 2. However, in line 897, only the ranking of `old_t1 + old_t2` is updated instead of considering `new_t1 + new_t2`.

<https://github.com/sherlock-audit/2025-05-layeredge/tree/main/edgen-staking/src/sake/LayerEdgeStaking.sol#L896>

```
function _checkBoundariesAndRecord(bool isRemoval) internal {
    ...
    // Tier 2 boundary handling
    if (new_t1 + new_t2 > 0) {
        if (new_t2 != old_t2) {
            // Need to update all users between the old and new tier 2
            boundaries
            uint256 old_boundary = old_t1 + old_t2;
            uint256 new_boundary = new_t1 + new_t2;

            if (new_boundary > old_boundary) {
                // Promotion case: update all users from old_boundary+1 to
                new_boundary
                for (uint256 rank = old_boundary + 1; rank <= new_boundary;
                rank++) {
                    _findAndRecordTierChange(rank, n);
                }
            } else {
                // Demotion case: update all users from new_boundary+1 to
                old_boundary
```

```

        for (uint256 rank = new_boundary + 1; rank <= old_boundary;
↪ rank++) {
            _findAndRecordTierChange(rank, n);
        }
    }
    // Handle case where Tier 2 count stays the same
    else if (isRemoval) {
        _findAndRecordTierChange(new_t1 + new_t2, n);
896:    } else if (!isRemoval) {
        _findAndRecordTierChange(old_t1 + old_t2, n);
    }
}
}

```

## Internal pre-conditions

N/A

## External pre-conditions

N/A

## Attack Path

Consider the scenario where `oldStakerCountInTree = 14` and `newStakerCountInTree = 15`. The tier distribution changes from (2, 4, 8) to (3, 4, 8), but the seventh-ranked user's tier does not change from tier 3 to tier 2.

## PoC

<https://github.com/sherlock-audit/2025-05-layeredge/tree/main/edgen-staking/test/stake/LayerEdgeStakingTest.t.sol>

```

function toString(uint256 value) internal pure returns (string memory result) {
    assembly {
        result := add(mload(0x40), 0x80)
        mstore(0x40, add(result, 0x20)) // Allocate memory.
        mstore(result, 0) // Zeroize the slot after the string.
        let end := result // Cache the end of the memory to calculate the length
↪ later.
        let w := not(0) // Tsk.
        for { let temp := value } 1 {} {
            result := add(result, w) // `sub(result, 1)`.
            mstore8(result, add(48, mod(temp, 10)))
        }
    }
}

```

```

        temp := div(temp, 10) // Keep dividing `temp` until zero.
        if iszero(temp) { break }
    }
    let n := sub(end, result)
    result := sub(result, 0x20) // Move the pointer 32 bytes back to make room
↪ for the length.
    mstore(result, n) // Store the length.
}
}
function test_LayerEdgeStaking_checkBoundariesAndRecord() public {
    vm.startPrank(admin);
    uint256 userAmount = 10_000 * 1e18;
    address[20] memory users;
    for (uint256 i = 0; i < 20; i++) {
        users[i] = makeAddr(string(abi.encodePacked("users", toString(i))));
        token.transfer(users[i], userAmount);
    }
    vm.stopPrank();
    for (uint256 i = 1; i <= 15; i++) {
        vm.startPrank(users[i]);
        token.approve(address(staking), MIN_STAKE);
        staking.stake(MIN_STAKE);
        vm.stopPrank();
    }
    address user = users[7];
    vm.warp(block.timestamp + 365 days);
    uint256 userAPY = staking.getUserAPY(user);
    vm.startPrank(user);
    uint256 beforeUserAmount = token.balanceOf(user);
    staking.claimInterest();
    uint256 afterUserAmount = token.balanceOf(user);
    console2.log("Procotol User   APY : %d", userAPY);
    console2.log("Actual Received APY : %d", 100e18 * (afterUserAmount -
↪ beforeUserAmount) / MIN_STAKE);
    vm.stopPrank();
}
}

```

forge test --match-test "test\_LayerEdgeStaking\_checkBoundariesAndRecord" -vv  
Result:

```

Ran 1 test for test/stake/LayerEdgeStakingTest.t.sol:LayerEdgeStakingTest
[PASS] test_LayerEdgeStaking_checkBoundariesAndRecord() (gas: 6634274)
Logs:
  Procotol User   APY : 3500000000000000000
  Actual Received APY : 2000000000000000000

```

After mitigation, Result:

```

Ran 1 test for test/stake/LayerEdgeStakingTest.t.sol:LayerEdgeStakingTest

```

```
[PASS] test_LayerEdgeStaking_checkBoundariesAndRecord() (gas: 6956510)
```

Logs:

Procotol User APY : 3500000000000000000

Actual Received APY : 3500000000000000000

## Impact

1. Some Users receive less interest.

## Mitigation

<https://github.com/sherlock-audit/2025-05-layeredge/tree/main/edgen-staking/src/sake/LayerEdgeStaking.sol#L897>

```
        if (new_t1 + new_t2 > 0) {
            if (new_t2 != old_t2) {
                // Need to update all users between the old and new tier 2
↪ boundaries
                uint256 old_boundary = old_t1 + old_t2;
                uint256 new_boundary = new_t1 + new_t2;

                if (new_boundary > old_boundary) {
                    // Promotion case: update all users from old_boundary+1 to
↪ new_boundary
                    for (uint256 rank = old_boundary + 1; rank <= new_boundary;
↪ rank++) {
                        _findAndRecordTierChange(rank, n);
                    }
                } else {
                    // Demotion case: update all users from new_boundary+1 to
↪ old_boundary
                    for (uint256 rank = new_boundary + 1; rank <= old_boundary;
↪ rank++) {
                        _findAndRecordTierChange(rank, n);
                    }
                }
            }
            // Handle case where Tier 2 count stays the same
            else if (isRemoval) {
                _findAndRecordTierChange(new_t1 + new_t2, n);
            } else if (!isRemoval) {
897:         _findAndRecordTierChange(old_t1 + old_t2, n);
+         _findAndRecordTierChange(new_t1 + new_t2, n);
            }
        }
```

# Issue M-1: stakerTierHistory is an unbound array that can be extended such that a user's funds are permanently lost

Source: <https://github.com/sherlock-audit/2025-05-layeredge-judging/issues/194>

## Found by

0x15, AestheticBhai, Artur, Bigsam, Boy2000, Cybrid, Drynooo, Galturok, KobbyEugene, Ollam, PersonaNormale, X77, ZafiN, algiz, eLSeR17, gesha17, illoy\_sci, jkoppel, lukris02, novaman33, ptsanev, sil3th, t0x1c, tjonair, tobi0x18, wickie

## Summary

In LayerEdgeStaking.sol, an stakerTierHistory is used for each user to track their tier history in a boundless array. This array is iterated through every time a user does any action with the protocol in calculateUnclaimedInterest(), which is called as part of \_updateInterest(), which is called as part of every user action. This array is iterated over in its entirety every time this happens, which can make any user operation eventually cost more gas than is permitted in a block, permanently locking the user's tokens in the contract.

## Root Cause

In calculateUnclaimedInterest(), the user's tier history is iterated through in its entirety.

```
for (uint256 i = 0; i < userTierHistory.length; i++) {
    if (userTierHistory[i].timestamp <= fromTime) {
        currentTier = userTierHistory[i].to;
        relevantStartIndex = i;
    } else {
        break;
    }
}
```

There is another loop which goes through the array starting at a given index:

```
for (uint256 i = relevantStartIndex + 1; i < userTierHistory.length; i++) {
```

This index is added to every time the user changes tier, which can happen because of other users staking and unstaking tokens. This happens in \_recordTierChange:

```
stakerTierHistory[user].push(TierEvent({from: old, to: newTier, timestamp:
↳   currentTime}));
```

This means that if a user sticks around in the staking pool long enough, organic user activity can lock his funds permanently. Additionally, an attacker can abuse the system by staking and unstaking small amounts over and over again to increase the length of a user's tier history array.

Additionally, every time the user's tier is iterated through the system, it also iterates through every apy change for that tier. This has a multiplicative effect on gas consumption, such that every apy change in the protocol's history will drastically decrease the number of tier changes a user needs to experience to be locked out of his funds.

## Internal Pre-conditions

The affected user must be staked and not be first in line.

## External Pre-conditions

No external preconditions.

## Attack Path

1. A user stakes a position
2. Whether because of organic activity or an attack, the user crosses the tier boundary several times
3. The user can now no longer withdraw

## Impact

The affected user loses all staked funds and can never withdraw.

## PoC

Please copy and paste this in LayerEdgeStakingTest.t.sol. Please run with an increased gas limit - forge test -vvvv --mt test\_audit\_gas\_bomb --gas-limit 1000000000000

```
function test_audit_gas_bomb() public {
    // NOTE must be run with high gas limit
    // forge test -vvvv --mt test_audit_gas_bomb --gas-limit 1000000000000
    address attacker = makeAddr("attackerAttacker");
    vm.startPrank(admin);
```

```

token.transfer(attacker, MIN_STAKE * 10000);

setupMultipleStakers(9);

vm.startPrank(attacker);
token.approve(address(staking), type(uint256).max);

for (uint256 i; i < 6000; i++) {
    address puppet = makeAddr(vm.toString(100001 + i));
    vm.startPrank(attacker);
    token.transfer(puppet, MIN_STAKE);
    vm.startPrank(puppet);
    token.approve(address(staking), MIN_STAKE);
    staking.stake(MIN_STAKE);
    staking.unstake(MIN_STAKE);
}

// bob's user history has been appended to so many times that he can't function
// any function he calls will revert because calculateUnclaimedInterest will
↪ take 45M gas
uint256 gasBefore = gasleft();
staking.getAllInfoOfUser(bob);
uint256 gasAfter = gasleft();
uint256 gasSpent = gasBefore - gasAfter;
console2.log(gasSpent); // according to the log, 87M gas spent, according to
↪ the trace, 45M
// either way, too much for bob to be able to do anything, his stake is stuck
}

```

## Mitigation

Consider storing a starting index for users so that the function doesn't iterate through every item every time. This would make it so that although the DOS is possible, as long as the user calls any function on the protocol every now and then the gas would remain within bounds. Currently, the whole thing is iterated over every time which given sufficient user activity can cause problems. Alternatively, allow users to collect interest with a function that processes n amount of tier changes so that the process can be broken into smaller transactions.

# Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.